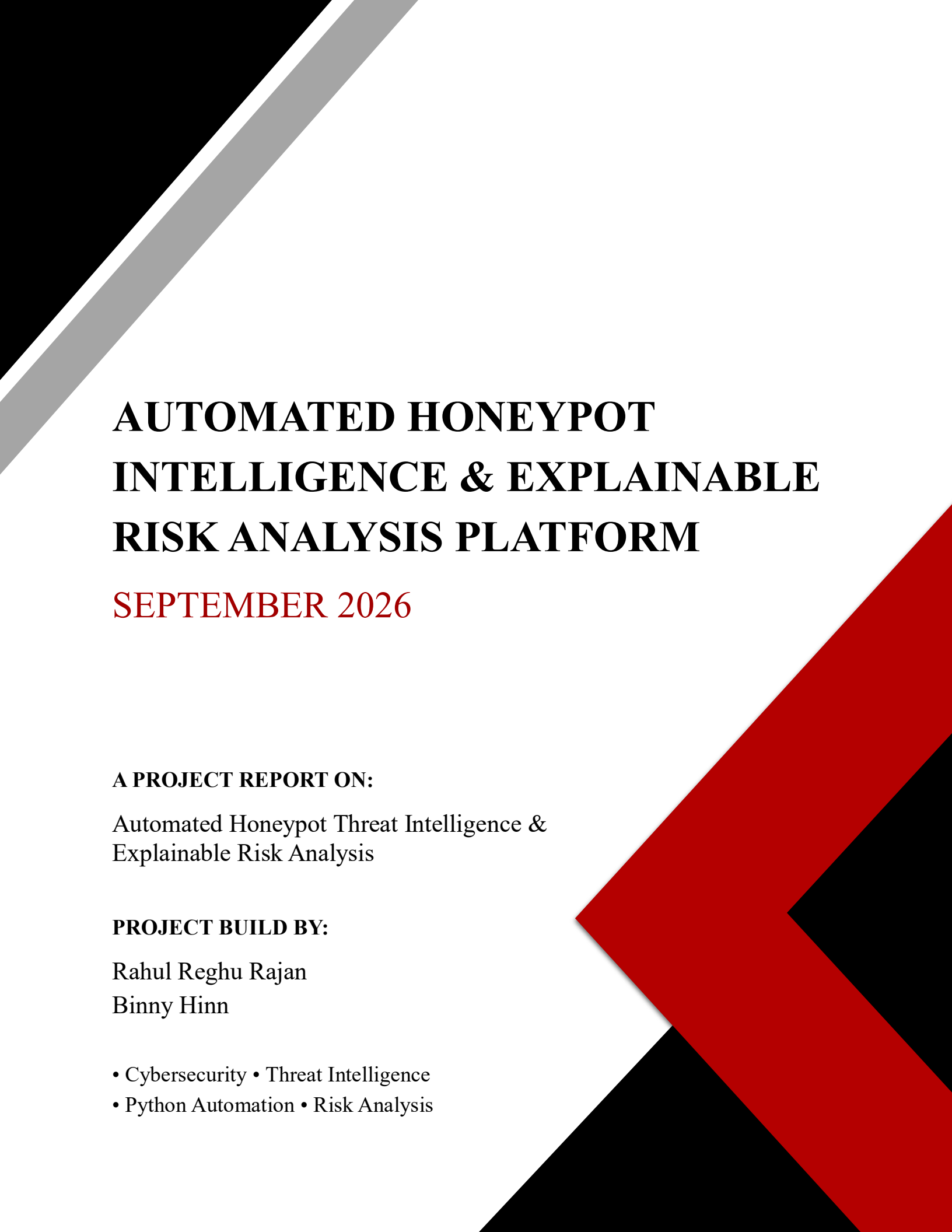




AUTOMATED HONEYPOT INTELLIGENCE & EXPLAINABLE RISK ANALYSIS PLATFORM

SEPTEMBER 2026

A PROJECT REPORT ON:

Automated Honeypot Threat Intelligence &
Explainable Risk Analysis

PROJECT BUILD BY:

Rahul Reghu Rajan
Binny Hinn

- Cybersecurity • Threat Intelligence
- Python Automation • Risk Analysis

DISCLAIMER

This document describes the design, implementation, testing, and evaluation of the Automated Honeypot Intelligence & Explainable Risk Analysis Platform, developed as a cybersecurity research and learning project. The platform is intended for authorized security research, controlled experimentation, defensive analysis, and educational purposes only. The honeypot environment is designed to operate within an isolated and controlled environment for observing and analyzing simulated or unauthorized interaction with the deployed decoy service.

The techniques, configurations, scripts, commands, and security concepts described in this document must only be used on systems, networks, applications, and infrastructure for which appropriate authorization has been obtained. The project does not constitute a production security monitoring, intrusion prevention, endpoint detection, or automated attack-blocking system. Results produced by the platform are intended to support security analysis and investigation and should not be treated as a substitute for professional security controls, human analysis, or organizational incident-response procedures. Threat intelligence results obtained from external services may change over time and may be subject to the accuracy, availability, limitations, and terms of the respective provider.

CONFIDENTIALITY NOTICE

This document may contain technical information relating to the project's implementation, configuration, testing environment, and security architecture.

The following information must not be disclosed in public versions of the documentation or repository:

- API keys and authentication credentials
- Passwords and secrets
- Private network information where disclosure is inappropriate
- SSH credentials
- Environment-specific sensitive configuration
- Raw attacker telemetry containing sensitive information
- Downloaded malicious or potentially malicious payloads
- Personally identifiable information
- Internal system identifiers or other operationally sensitive data

Publicly shared versions of the project should contain only sanitized configuration, source code, screenshots, logs, and test results that do not expose credentials or sensitive operational information.

Actual API credentials are stored outside the source code through environment-based configuration and are not intended to be committed to version control.

TABLE OF CONTENTS

1. Introduction	01
2. System overview & architecture	03
3. Honeypot deployment & telemetry collection	06
4. Automated log ingestion & event normalization	12
5. IOC extraction & behavioral analysis	14
6. Threat intelligence integration	16
7. Explainable risk scoring engine	19
8. Incident correlation & automated reporting.....	23
9. Security analyst dashboard	25
10. Testing, validation & results	30
11. Conclusion	32
12. References	33
Appendix A – Contact Information	34

1. INTRODUCTION

The increasing volume and complexity of cyber threats have made automated security monitoring, threat intelligence, and efficient incident analysis important components of modern cybersecurity operations. Security systems can generate large amounts of telemetry from authentication attempts, command execution, network activity, and other attacker interactions. However, raw security logs alone may not provide sufficient context for quickly understanding attacker behavior, identifying indicators of compromise, assessing risk, and prioritizing suspicious activity.

Honeypots provide a controlled method for observing unauthorized interactions with intentionally exposed services. Instead of protecting a production service, a honeypot is designed to attract and record suspicious activity, providing valuable telemetry that can be analyzed to understand attacker techniques and behavior. This project uses the Cowrie SSH Honeypot as the telemetry source and builds an automated analysis pipeline around the activity it generates.

The Automated Honeypot Intelligence & Explainable Risk Analysis Platform is designed to transform raw Cowrie telemetry into structured security intelligence. The implemented platform automates multiple stages of analysis, beginning with log ingestion and event normalization, followed by IOC extraction, authentication and behavioral analysis, threat intelligence enrichment through AbuseIPDB, explainable risk scoring, incident correlation, automated reporting, and visualization through a Streamlit-based security analyst dashboard.

The primary objective of the project is to reduce the amount of manual processing required to interpret honeypot activity while maintaining a clear connection between observed attacker actions and the resulting security assessment. The platform stores processed security information in SQLite, allowing events, sessions, indicators, threat intelligence results, risk assessments, and correlated incidents to be retained for analysis.

The project covers the complete analytical workflow from attacker interaction to analyst-facing intelligence:

Attacker Interaction



Cowrie SSH Honeypot



Log Ingestion & Normalization



IOC & Behavioral Analysis



Threat Intelligence Enrichment



Explainable Risk Scoring



Incident Correlation



Automated Incident Report



Security Analyst Dashboard

The project is implemented as a controlled cybersecurity research and learning environment. Its scope is focused on telemetry collection, automated analysis, threat intelligence, risk prioritization, incident correlation, reporting, and visualization. It does not attempt to function as a complete enterprise SIEM, endpoint detection platform, intrusion prevention system, or automated attack-blocking solution.

The completed implementation demonstrates how multiple cybersecurity analysis techniques can be integrated into a single automated workflow, providing a practical foundation for understanding honeypot telemetry, threat intelligence processing, security automation, risk analysis, and analyst-oriented incident investigation.

2. SYSTEM OVERVIEW & ARCHITECTURE

The Automated Honeypot Intelligence & Explainable Risk Analysis Platform is designed as an automated security monitoring and threat analysis system that transforms attacker activity captured by a honeypot into structured security intelligence, risk assessments, and analyst-readable incidents. The platform combines a Cowrie SSH honeypot, automated Python-based processing components, external Threat Intelligence, a centralized SQLite database, incident reporting, and a Streamlit-based security analyst dashboard.

The system operates by first exposing the Cowrie honeypot to controlled network activity. When an attacker or simulated attacker connects to the SSH service, Cowrie records the interaction as telemetry, including connection information, authentication attempts, commands, sessions, and other relevant activity. The collected telemetry forms the primary input for the automated analysis pipeline.

The architecture follows a sequential security-analysis workflow:

Attacker Activity → Cowrie Honeypot → Log Ingestion → Event Normalization → IOC & Behavioral Analysis → Threat Intelligence → Risk Scoring → Incident Correlation → Database → Dashboard & Reports

The design separates telemetry collection, security analysis, data storage, and visualization so that each stage performs a specific function. This allows raw honeypot events to be progressively transformed into meaningful security information rather than presenting unprocessed logs directly to an analyst.

System Architecture

The honeypot operates within the Ubuntu environment and listens for SSH connections on the configured honeypot service. In the implemented environment, Cowrie telemetry is generated from attacker interactions and stored in its log directory. The configuration of the honeypot environment is maintained through `etc/cowrie.cfg`.

The collected logs are processed by the automation layer. `ingest.py` is responsible for obtaining relevant Cowrie events, while `normalize.py` converts the different raw event structures into a consistent internal representation. This normalized data provides a common format for subsequent analysis.

The next stage performs security analysis. `ioc_extract.py` identifies indicators such as IP addresses and other observable values from the collected events. `behavior.py` analyzes attacker activity and interaction patterns to identify meaningful behavioral characteristics. These results are passed to

the Threat Intelligence layer, where `threat_intel.py` and `ioc_ti_pipeline.py` perform external intelligence enrichment, including reputation-based analysis using services such as AbuseIPDB when available.

The enriched security information is then evaluated by the `risk_engine.py` component. Instead of producing only a numerical score, the risk engine provides an explainable assessment based on contributing factors observed during analysis. This allows the resulting severity to be understood in terms of the evidence that influenced the assessment.

Processed events, indicators, intelligence results, risk assessments, sessions, and incidents are maintained in the centralized SQLite database through `database.py`. The database acts as the persistent data layer for the platform and allows information generated by different stages of the pipeline to be correlated and retrieved consistently.

The incident-management stage uses the stored information to identify related attacker activity and produce structured incident information. `incident_report.py` supports automated incident reporting, while the resulting information is presented through the Streamlit-based `dashboard/app.py` interface. The dashboard provides the security analyst with a consolidated view of events, sessions, indicators, risk information, and correlated incidents.

External Threat Intelligence is represented as an enrichment layer rather than a primary source of telemetry. This means that the platform can continue processing locally collected honeypot activity even when external intelligence is unavailable, while available reputation information can improve the context and confidence of the resulting analysis.

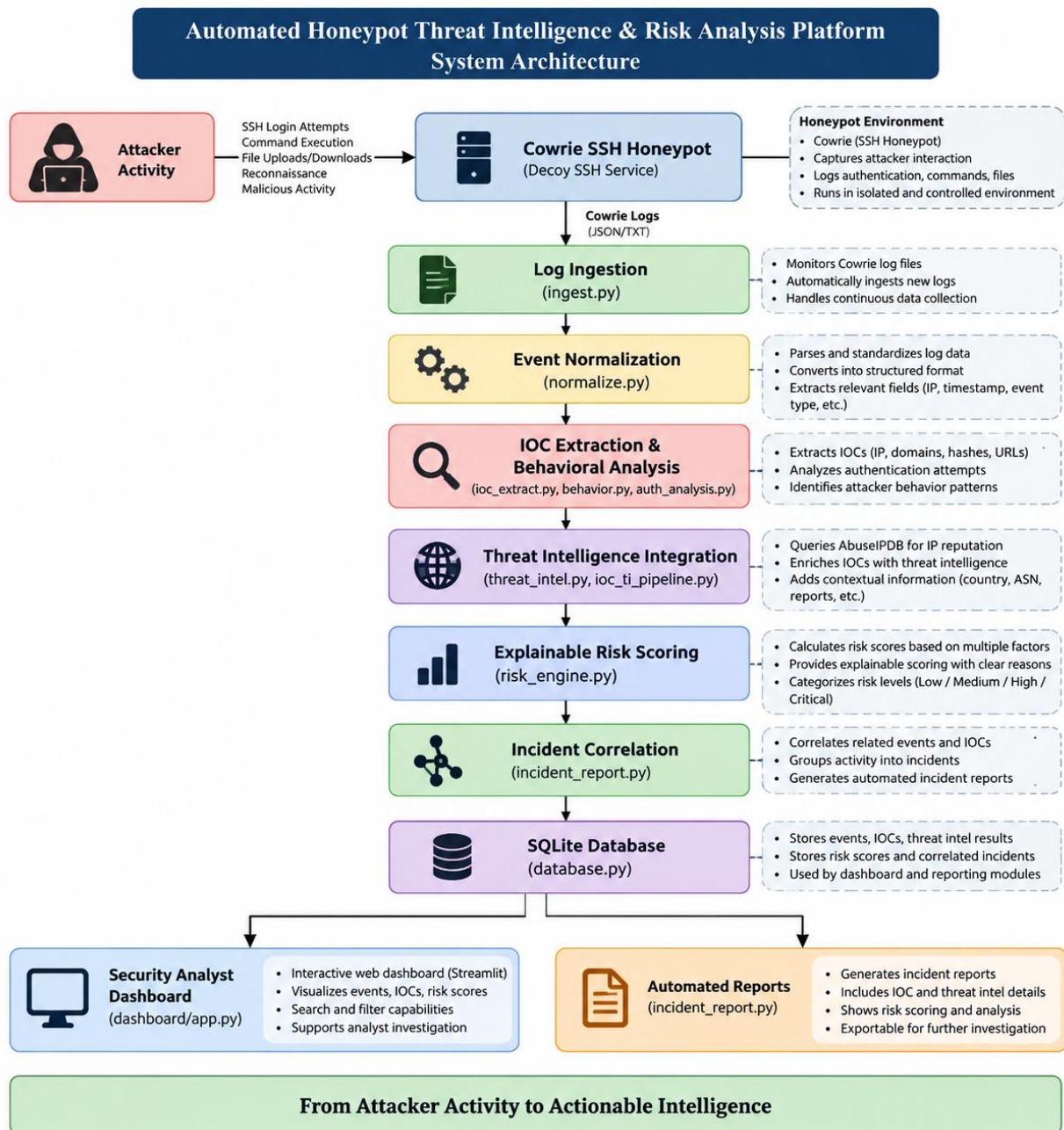


Figure 1: System Architecture of the Automated Honeypot Threat Intelligence & Risk Analysis Platform

3. HONEYPOT DEPLOYMENT & TELEMETRY COLLECTION

A honeypot is a deliberately deployed system designed to attract and observe malicious activity. Unlike a production server, its primary purpose is not to provide services to legitimate users, but to collect information about attacker behavior, techniques, and interactions. In this project, an SSH-based honeypot environment was deployed using Cowrie to capture unauthorized connection attempts and generate security telemetry for further analysis.

Cowrie is an open-source SSH and Telnet honeypot that simulates an interactive system environment. When an attacker attempts to connect to the exposed SSH service, the honeypot records the interaction instead of providing access to the actual host operating system. This allows suspicious authentication attempts, commands, connection details, and attacker behavior to be safely monitored and collected.

The deployed honeypot acts as the primary telemetry source for the Automated Honeypot Intelligence & Explainable Risk Analysis Platform. The collected events form the starting point of the complete security analysis pipeline. Once generated, the telemetry is processed through automated ingestion, normalization, IOC extraction, behavioral analysis, threat intelligence enrichment, risk scoring, and incident reporting.

3.1 Cowrie Installation and Startup

Cowrie was deployed on the Ubuntu-based environment and configured to operate as the SSH honeypot service. The deployment environment was prepared with the required dependencies and Python environment before starting the Cowrie service.

The Cowrie configuration is maintained through the following important configuration file:

`etc/cowrie.cfg`

This file contains the configuration settings required for the honeypot environment and controls the operational behavior of the Cowrie service.

The Cowrie service was started from its installation directory using the configured Python environment. Once running, the honeypot was ready to receive and record SSH interaction attempts.

```
cowrie@ahtra: ~/cowrie
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> ssh cipherx@192.168.1.14
cipherx@192.168.1.14's password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 7.0.0-31-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep 15 05:21:45 AM UTC 2026

System load:  1.14          Processes:      127
Usage of /:   54.0% of 11.21GB Users logged in:    0
Memory usage: 16%          IPv4 address for enp0s8: 192.168.1.14
Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.
   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Mon Sep 14 10:16:52 2026 from 192.168.1.5
cipherx@ahtra:~$ sudo -iu cowrie
[sudo: authenticate] Password:
cowrie@ahtra:~$ cd ~/cowrie
cowrie@ahtra:~/cowrie$ source cowrie-env/bin/activate
(cowrie-env) cowrie@ahtra:~/cowrie$ cowrie start
Removed stale PID file var/run/cowrie.pid
Starting cowrie: [twistd --umask=0022 --pidfile var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie]...
(cowrie-env) cowrie@ahtra:~/cowrie$ cowrie status
cowrie is running (PID: 1794).
(cowrie-env) cowrie@ahtra:~/cowrie$ ss -tulnp | grep 2222
tcp    LISTEN 0      50          0.0.0.0:2222    0.0.0.0:*      users:((("twistd",pid=1794,fd=12))
(cowrie-env) cowrie@ahtra:~/cowrie$
```

Figure 2: Cowrie Honeypot Installation / Startup in Ubuntu Terminal

3.2 SSH Honeypot Operation

The deployed Cowrie instance simulates an SSH service that can receive connection attempts from external systems within the testing environment. When a connection is made to the honeypot, the interaction is handled inside the simulated environment rather than granting access to the actual Ubuntu operating system.

This approach provides a controlled environment for observing potentially malicious activity. Attackers or testing systems can attempt authentication, execute commands, and interact with the simulated shell while Cowrie records the generated telemetry.

The honeypot deployment therefore provides visibility into activities such as:

- SSH connection attempts
- Source IP addresses
- Authentication attempts
- Username and password activity
- Commands entered during sessions
- Session creation and termination
- Download or execution-related activity where captured by Cowrie

These events provide valuable raw data for identifying suspicious behavior and building an automated security analysis workflow.

3.3 Attacker Interaction and Telemetry Generation

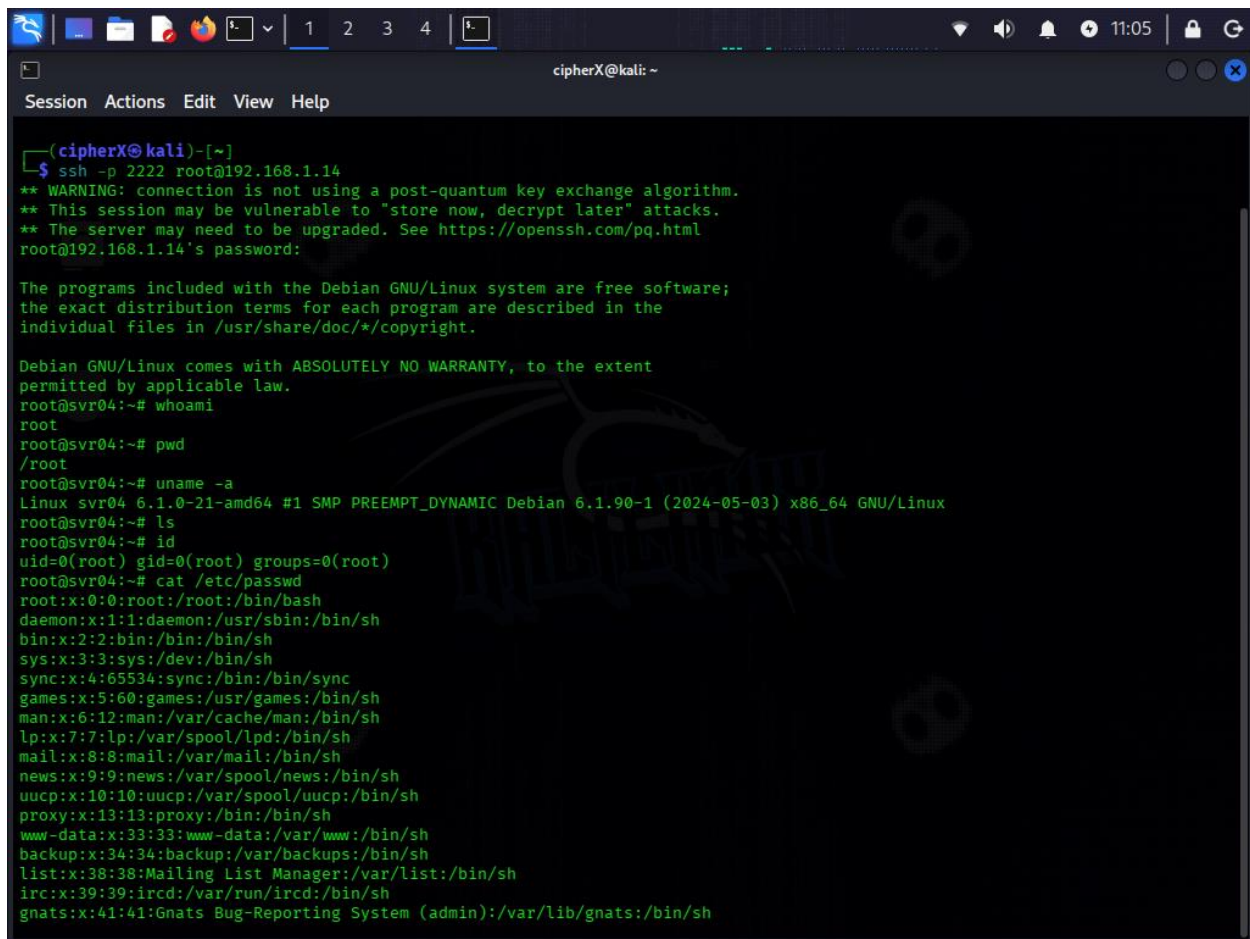
When an attacker interacts with the SSH honeypot, Cowrie generates detailed event records describing the activity. Each interaction contributes to the telemetry collected by the platform.

For example, repeated login attempts can indicate credential-based attacks, while unusual commands entered after a successful interaction can provide insight into attacker objectives and behavior.

The collected telemetry allows the system to answer important security questions such as:

- Who attempted to connect?
- When did the activity occur?
- Which usernames were targeted or used?
- How many authentication attempts were made?
- What commands were executed?
- Did the behavior appear suspicious or malicious?

Rather than manually investigating each event, this project uses the generated telemetry as input for the automated analysis pipeline developed in the later stages of the platform.



```
(cipherX@kali)-[~]
$ ssh -p 2222 root@192.168.1.14
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@192.168.1.14's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# whoami
root
root@svr04:~# pwd
/root
root@svr04:~# uname -a
Linux svr04 6.1.0-21-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.90-1 (2024-05-03) x86_64 GNU/Linux
root@svr04:~# ls
root@svr04:~# id
uid=0(root) gid=0(root) groups=0(root)
root@svr04:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
```

Figure 3: SSH Attacker Interaction with the Cowrie Honeypot

3.4 Log Sources and Telemetry Format

Cowrie records honeypot activity in structured log files. These logs act as the primary data source for the automation components implemented in this project.

The telemetry generated by Cowrie contains information related to connection events, authentication attempts, commands, sessions, and other attacker interactions. Structured event data makes it possible for Python-based automation scripts to read and process the information programmatically.

The automated pipeline uses these logs as the starting point for security analysis.

The major processing flow is:

Cowrie Honeypot → Security Logs → Automated Ingestion → Event Normalization → Security Analysis

The log data collected at this stage is intentionally preserved as raw telemetry before being processed by the automation pipeline. This separation allows the system to maintain a clear workflow between data collection and security analysis.

```
cowrie@ahkira: ~/cowrie
(cowrie-env) cowrie@ahkira:~/cowrie$ tail -n 15 var/log/cowrie/cowrie.json
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "hash": "eeca2460550b0ded084ecf2f70a75356", "hashAlgorithms": "mlk", "keyAlgorithms": "mlkem768x25519-sha256", "curve25519-sha256", "curve25519-sha256@libssh.org", "ecdhe-sha2-nistp256", "ecdhe-sha2-nistp384", "ecdhe-sha2-nistp521", "diffie-hellman-group-exchange-sha256", "diffie-hellman-group18-sha512", "diffie-hellman-group18-sha512", "diffie-hellman-group14-sha256", "ext-info-c", "kex-strict-c-v00@openssh.com", "chacha20-poly1305@openssh.com", "aes128-gcm@openssh.com", "aes256-gcm@openssh.com", "aes128-ctr", "aes192-ctr", "aes256-ctr", "umac-64-etm@openssh.com", "umac-128-etm@openssh.com", "hmac-sha2-256-etm@openssh.com", "hmac-sha2-512-etm@openssh.com", "hmac-sha1-etm@openssh.com", "umac-64@openssh.com", "umac-128@openssh.com", "hmac-sha2-256", "hmac-sha2-512", "hmac-sha1", "none", "zlib@openssh.com", "kexAlgs": ["mlkem768x25519-sha256", "sntrup761x25519-sha512", "sntrup761x25519-sha512@openssh.com", "curve25519-sha256", "curve25519-sha256@libssh.org", "ecdhe-sha2-nistp256", "ecdhe-sha2-nistp384", "ecdhe-sha2-nistp521", "diffie-hellman-group-exchange-sha256", "diffie-hellman-group18-sha512", "diffie-hellman-group18-sha512", "diffie-hellman-group14-sha256", "ext-info-c", "kex-strict-c-v00@openssh.com", "keyAlgs": ["ssh-ed25519-cert-v01@openssh.com", "ecdsa-sha2-nistp256-cert-v01@openssh.com", "ecdsa-sha2-nistp384-cert-v01@openssh.com", "ecdsa-sha2-nistp521-cert-v01@openssh.com", "sk-ssh-ed25519-cert-v01@openssh.com", "sk-ecdsa-sha2-nistp256-cert-v01@openssh.com", "webauthn-sk-ecdsa-sha2-nistp256-cert-v01@openssh.com", "rsa-sha2-512-cert-v01@openssh.com", "rsa-sha2-256-cert-v01@openssh.com", "ssh-ed25519", "ecdsa-sha2-nistp256", "ecdsa-sha2-nistp384", "ecdsa-sha2-nistp521", "sk-ssh-ed25519@openssh.com", "sk-ecdsa-sha2-nistp256@openssh.com", "webauthn-sk-ecdsa-sha2-nistp256@openssh.com", "rsa-sha2-512", "rsa-sha2-256"], "encCS": ["chacha20-poly1305@openssh.com", "aes128-gcm@openssh.com", "aes256-gcm@openssh.com", "aes128-ctr", "aes192-ctr", "aes256-ctr"], "macCS": ["umac-64-etm@openssh.com", "umac-128-etm@openssh.com", "hmac-sha2-256-etm@openssh.com", "hmac-sha2-512-etm@openssh.com", "hmac-sha1-etm@openssh.com", "umac-64@openssh.com", "umac-128@openssh.com", "hmac-sha2-256", "hmac-sha2-512", "hmac-sha1"], "compCS": ["none", "zlib@openssh.com"], "langCS": [""], "eventid": "cowrie.client.kex", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:24.953849Z", "message": "SSH client hash fingerprint: eeca2460550b0ded084ecf2f70a75356"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "username": "root", "password": "1234", "eventid": "cowrie.login.succes", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:30.390604Z", "message": "Login attempt [root/1234] succeeded"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "width": 124, "height": 41, "eventid": "cowrie.client.size", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:30.419536Z", "message": "Terminal Size: 124 41"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "name": "COLORTERM", "value": "truecolor", "eventid": "cowrie.client.v", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:30.419798Z", "message": "request_env: COLORTERM=truecolor"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "name": "LANG", "value": "en_IN", "eventid": "cowrie.client.var", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:30.422475Z", "message": "request_env: LANG=en_IN"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "whoami", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:49.284976Z", "message": "CMD: whoami"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "pwd", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:53.002784Z", "message": "CMD: pwd"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "uname -a", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:33:58.972036Z", "message": "CMD: uname -a"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "ls", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:34:02.857639Z", "message": "CMD: ls"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "id", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:34:04.558638Z", "message": "CMD: id"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "cat /etc/passwd", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:34:27.507227Z", "message": "CMD: cat /etc/passwd"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "input": "exit", "eventid": "cowrie.command.input", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:36:17.794623Z", "message": "CMD: exit"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "tylog": "var/lib/cowrie/tty/0bde0e201c597e97ff102bd8c30ad98a3868083298647d59e2571d225679ba9b", "size": 1563, "shasum": "0bde0e201c597e97ff102bd8c30ad98a3868083298647d59e2571d225679ba9b", "duplicate": false, "duration_ms": 167380, "eventid": "cowrie.log.closed", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:36:17.800228Z", "message": "Closing TTY log: var/lib/cowrie/tty/0bde0e201c597e97ff102bd8c30ad98a3868083298647d59e2571d225679ba9b after 167380 milliseconds"}
{"session": "085d53813f46", "protocol": "ssh", "src_ip": "192.168.1.10", "src_port": 37906, "dst_ip": "192.168.1.14", "dst_port": 2222, "duration_ms": 173116, "eventid": "cowrie.session.closed", "sensor": "ahkira", "uuid": "765734a0-aad3-11f1-a5db-0800274106d0", "timestamp": "2026-09-15T05:36:17.816625Z", "message": "Connection lost after 173116 milliseconds"}
(cowrie-env) cowrie@ahkira:~/cowrie$
```

Figure 4: Cowrie Telemetry / Log Events Generated by Attacker Activity

3.5 Role of Honeypot Telemetry in the Platform

The honeypot is the foundation of the entire platform because all later security analysis depends on the telemetry generated during attacker interaction.

Once Cowrie produces the security events, the automation layer processes them through multiple stages:

1. Log ingestion
2. Event normalization
3. Authentication and behavioral analysis
4. IOC extraction
5. Threat intelligence enrichment
6. Explainable risk scoring
7. Incident correlation
8. Automated reporting
9. Security analyst dashboard visualization

This architecture transforms raw honeypot events into meaningful security intelligence rather than leaving the collected logs for manual analysis. The telemetry collection stage therefore establishes the connection between the deployed cybersecurity infrastructure and the automated intelligence components of the project.

4. AUTOMATED LOG INGESTION & EVENT NORMALIZATION

The effectiveness of a honeypot-based security monitoring system depends not only on collecting attacker activity but also on processing the generated telemetry in a structured and usable form. Raw Cowrie logs contain valuable information about authentication attempts, session activity, commands, source IP addresses, and other attacker interactions. However, raw logs are not always suitable for direct analysis because different events may contain different fields and formats.

To address this, the platform implements an automated log ingestion and event normalization pipeline. This stage continuously processes honeypot telemetry and converts relevant security events into a standardized structure that can be used by later components such as IOC extraction, behavioral analysis, threat intelligence enrichment, risk scoring, incident correlation, and dashboard visualization.

The automated ingestion process is implemented using `automation/ingest.py`, while event standardization is handled through `automation/normalize.py`.

4.1 Automated Log Ingestion

The ingestion component reads security events generated by the Cowrie honeypot and processes new log entries automatically. Instead of manually reviewing large log files, the system extracts relevant events from the available telemetry and forwards them through the automation pipeline.

The ingestion process helps the system:

- Continuously process new honeypot events.
- Identify relevant attacker activity.
- Reduce manual log analysis.
- Prepare security events for further processing.
- Support automated downstream security analysis.

This automation transforms Cowrie from a simple interaction logging system into the primary telemetry source of the larger threat intelligence and risk analysis platform.

4.2 Event Normalization

After ingestion, security events are normalized into a consistent structure. Normalization ensures that important information is represented in a predictable format even when different event types contain different fields.

Important information processed during normalization includes:

- Source IP address
- Event type
- Timestamp
- Authentication activity
- Commands executed
- Session-related information
- Other available attacker interaction data

The normalized event structure provides a common foundation for the remaining automation pipeline. This is particularly important because later components should not need to repeatedly interpret raw Cowrie log formats.

4.3 Database Storage

Processed and normalized security events are also prepared for structured storage through the platform's database component, implemented in `automation/database.py`. Database storage allows collected security information to be retained and accessed by other components of the system instead of depending entirely on raw log files.

This supports:

- Historical event tracking
- Incident correlation
- Dashboard visualization
- Risk analysis
- Investigation of attacker activity over time

5. IOC EXTRACTION & BEHAVIORAL ANALYSIS

The IOC Extraction and Behavioral Analysis module is responsible for examining the telemetry collected from the deployed honeypot and identifying indicators that may represent malicious or suspicious activity. The main purpose of this stage is to convert raw attacker activity recorded in honeypot logs into structured security information that can be further analysed using threat intelligence and risk assessment techniques.

The module processes the collected logs and searches for commonly observed Indicators of Compromise (IOCs), including IP addresses, domain names, URLs, file hashes, usernames, commands, ports, and other suspicious artefacts. These indicators provide useful information about the infrastructure and techniques used by attackers while interacting with the honeypot.

The extracted IOCs are separated from normal log information so that they can be analysed independently. For example, an external IP address observed during an SSH login attempt can be extracted as an IOC, while the associated username, password attempt, command, timestamp, and session information can be retained as behavioural evidence. This allows the system to study not only *who* is attacking the honeypot, but also *how* the attacker behaves after gaining access.

The behavioural analysis component examines patterns in attacker activity such as repeated login attempts, commonly used usernames, password-guessing behaviour, executed shell commands, connection attempts to different ports, downloading of files, and other suspicious actions. Repeated events from the same source can be grouped together to identify persistent or automated attacks. Command sequences can also provide an indication of the attacker's objective, such as system discovery, privilege escalation, persistence, malware download, or network reconnaissance.

The extracted indicators can then be passed to the Threat Intelligence pipeline for enrichment. External threat intelligence information can be used to determine whether an IP address, domain, hash, or URL has previously been associated with malicious activity. Combining extracted IOCs with behavioural evidence provides a stronger basis for evaluating the severity of an observed attack than relying on a single indicator alone.

The implementation is supported by the project's IOC extraction and threat-intelligence processing components, including `ioc_extract.py` and `ioc_ti_pipeline.py`. The IOC extraction component identifies and structures relevant indicators from the collected telemetry, while the threat-intelligence pipeline processes these indicators for enrichment and analysis.

The output of this stage provides a structured representation of attacker activity that can be used by the subsequent risk analysis module. This creates a continuous flow from raw honeypot telemetry → IOC extraction → behavioural analysis → threat intelligence enrichment → risk assessment. Therefore, the module acts as an important bridge between basic log collection and automated security analysis.

Output Proof

The following screenshot shows the output generated by the IOC Extraction and Behavioral Analysis component during project execution. The output provides evidence that the system successfully processes honeypot telemetry and identifies relevant security indicators and behavioural information.

```
cowrie@ahtra: ~/cowrie
(cowrie-env) cowrie@ahtra:~/cowrie$ python automation/ioc_extract.py
{"timestamp": "2026-09-15T05:33:24.699906Z", "event_type": "cowrie.session.connect", "session_id": "085d53813f46", "source_ip": "192.168.1.10", "iocs": {"ips": ["192.168.1.10", "192.168.1.14"], "urls": [], "domains": [], "hashes": []}}
{"timestamp": "2026-09-15T05:33:24.953849Z", "event_type": "cowrie.client.kex", "session_id": "085d53813f46", "source_ip": "192.168.1.10", "iocs": {"ips": [], "urls": [], "domains": [], "hashes": ["eeca2460550b9ded084ecf2f70a75356"]}}
{"timestamp": "2026-09-15T05:36:17.800220Z", "event_type": "cowrie.log.closed", "session_id": "085d53813f46", "source_ip": "192.168.1.10", "iocs": {"ips": [], "urls": [], "domains": [], "hashes": ["0bde0e201c597e97ff102bd8c38ad98a3868083298647d59e2571d225679ba9b"]}}
(cowrie-env) cowrie@ahtra:~/cowrie$ python automation/behavior.py

=====
COWRIE BEHAVIOR ANALYSIS ENGINE
=====

BEHAVIOR SUMMARY
=====
{
  "total_events": 17,
  "authentication_events": 1,
  "brute_force_sources": 0,
  "reconnaissance_events": 6,
  "network_reconnaissance_events": 0,
  "suspicious_command_events": 0,
  "payload_activity_events": 0,
  "persistence_attempts": 1
}

BRUTE FORCE
=====
No brute-force activity detected.

AUTHENTICATION
=====
{
  "timestamp": "2026-09-15T05:33:30.390604Z",
  "source_ip": "192.168.1.10",
  "session_id": "085d53813f46",
  "username": "root",
  "behavior": "SUCCESSFUL_AUTHENTICATION"
}

RECONNAISSANCE
=====
{
  "timestamp": "2026-09-15T05:33:49.284976Z",
  "source_ip": "192.168.1.10",
  "session_id": "085d53813f46",
  "command": "whoami",
  "behavior": "RECONNAISSANCE"
}
{
  "timestamp": "2026-09-15T05:33:53.002784Z",
```

Figure 5: IOC Extraction & Behavioral Analysis Output

6. THREAT INTELLIGENCE INTEGRATION

Threat Intelligence Integration enriches the Indicators of Compromise (IOCs) extracted from the honeypot with information from external security intelligence sources. The purpose of this module is to determine whether an observed IP address has previously been associated with malicious or abusive activity. This provides additional context to the raw honeypot logs and improves automated threat analysis.

In this project, the extracted IOCs are passed to the Threat Intelligence Pipeline, mainly implemented using `ioc_ti_pipeline.py`. The pipeline communicates with external threat-intelligence services through APIs, retrieves reputation information, and prepares the results for the risk-analysis stage.

6.1 AbuseIPDB Integration

The primary threat-intelligence service used in the project is AbuseIPDB. It provides reputation information about IP addresses reported for abusive or malicious activity. When an attacker IP is identified from the honeypot logs, the system sends the IP address to the AbuseIPDB API and processes the returned information.

Important information obtained from the lookup includes:

- IP address
- Abuse Confidence Score
- Number of reports
- Country
- ISP or network information
- Domain and usage information, when available
- Last reported activity

The Abuse Confidence Score is an important indicator used during analysis. A higher score represents stronger evidence that the IP has been associated with abusive activity. However, the score is not considered absolute proof of an attack. It is combined with the behaviour observed by the honeypot.

6.2 Threat Intelligence Enrichment

Threat intelligence enrichment means adding external information to an IOC collected from the local honeypot environment. For example, if the honeypot records repeated SSH login attempts from an IP address, the system can query that IP through AbuseIPDB and associate its reputation with the observed event.

The implemented workflow is:

Honeypot Logs → IOC Extraction → IOC Processing → AbuseIPDB Lookup → Intelligence Enrichment → Risk Analysis

The honeypot provides local evidence such as login attempts, failed authentication, commands, connection frequency, and other attacker behaviour. AbuseIPDB provides external reputation information. Combining both sources gives a stronger basis for evaluating the severity of an event.

6.3 API Processing and Security

The integration uses API-based communication to automate the reputation lookup. The pipeline prepares the request, sends the extracted IP address, receives the API response, and extracts the required fields for further processing.

The system should also handle network failures, invalid responses, unavailable services, and API limits without stopping the complete analysis process. If an external lookup fails, the original IOC and honeypot behaviour can still be retained.

The AbuseIPDB API key should be stored securely using an environment variable or configuration file rather than being directly written into the source code. This prevents sensitive credentials from being exposed when the project is maintained or uploaded to GitHub.

6.4 IOC Reputation and Correlation

Threat intelligence becomes more useful when external reputation is correlated with honeypot behaviour. For example, an IP that repeatedly attempts SSH authentication is suspicious based on its local behaviour. If the same IP also has a high AbuseIPDB confidence score and multiple reports, the combined evidence provides stronger support for identifying the source as potentially malicious.

The platform therefore combines:

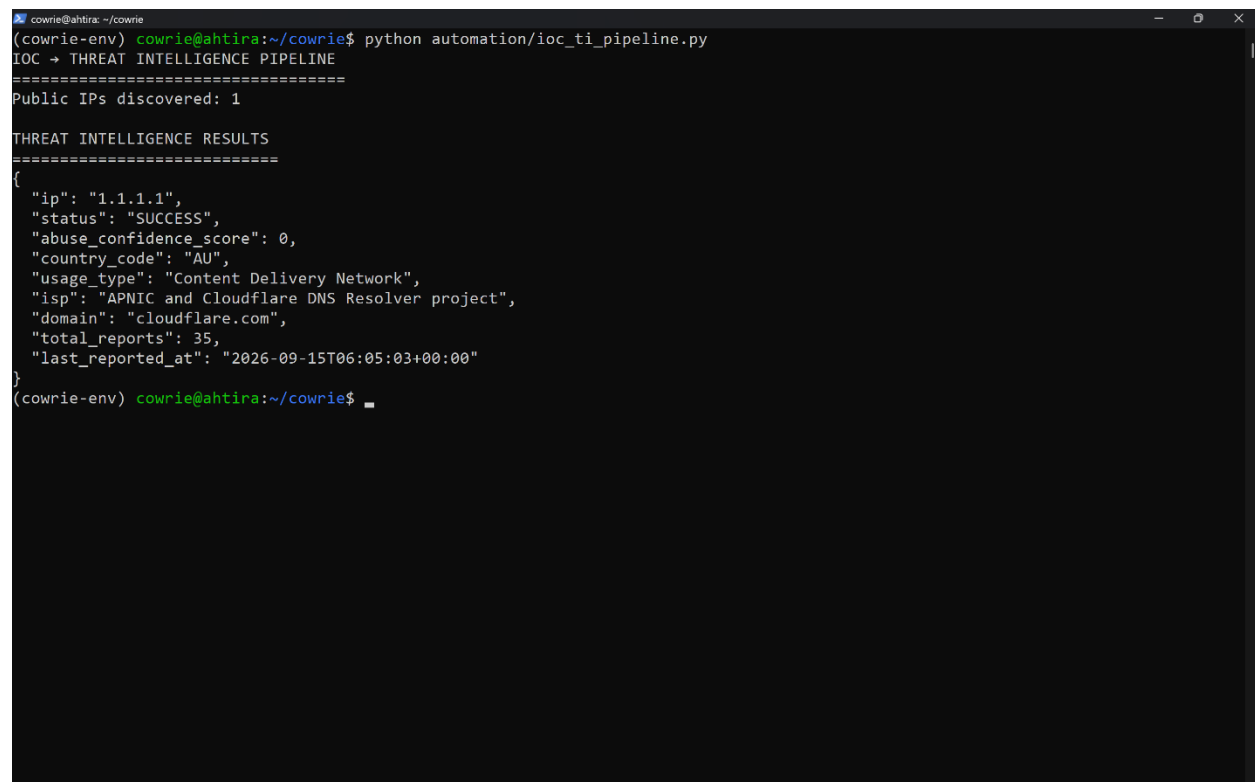
Honeypot Evidence: source IP, login attempts, failed authentication, commands, and repeated activity.

Threat Intelligence: abuse confidence, number of reports, IP reputation, country, and network information.

Although AbuseIPDB is the primary source implemented in the project, the architecture can be extended to other intelligence sources for domain reputation, URL reputation, file hashes, malware information, and attacker TTPs.

6.5 Role in the Overall System

Threat Intelligence Integration acts as the link between IOC Extraction and Risk Analysis. IOC extraction identifies suspicious indicators from the honeypot, while threat intelligence provides additional information about their reputation. The enriched data is then passed to the risk-analysis module, where it is combined with attacker behaviour to determine the threat level and priority.



```
cowrie@ahtra: ~/cowrie
(cowrie-env) cowrie@ahtra:~/cowrie$ python automation/ioc_ti_pipeline.py
IOC → THREAT INTELLIGENCE PIPELINE
=====
Public IPs discovered: 1

THREAT INTELLIGENCE RESULTS
=====
{
  "ip": "1.1.1.1",
  "status": "SUCCESS",
  "abuse_confidence_score": 0,
  "country_code": "AU",
  "usage_type": "Content Delivery Network",
  "isp": "APNIC and Cloudflare DNS Resolver project",
  "domain": "cloudflare.com",
  "total_reports": 35,
  "last_reported_at": "2026-09-15T06:05:03+00:00"
}
(cowrie-env) cowrie@ahtra:~/cowrie$
```

Figure 6: Threat Intelligence Integration Output

7. EXPLAINABLE RISK SCORING ENGINE

The Explainable Risk Scoring Engine is responsible for converting the collected security evidence into an understandable risk score and threat level. Instead of simply classifying an IP address as malicious or safe, the engine considers multiple factors such as attacker behaviour, IOC reputation, activity frequency, and threat-intelligence results. The main purpose of this module is to provide a clear reason behind the assigned risk level. This makes the analysis easier to understand and helps security analysts prioritize suspicious activities.

The risk-scoring functionality is implemented using the project's analysis and risk-processing code, where the information obtained from honeypot telemetry, IOC extraction, and threat-intelligence enrichment is combined.

7.1 Risk Factors

The engine considers several indicators when calculating the overall risk:

- **IOC Reputation** – reputation obtained from threat-intelligence services such as AbuseIPDB.
- **Attack Frequency** – repeated connection or login attempts from the same source.
- **Authentication Activity** – repeated failed login attempts or suspicious authentication behaviour.
- **Command Activity** – potentially malicious commands executed after gaining access.
- **Network Behaviour** – suspicious ports, repeated connections, scanning, or unusual traffic.
- **Threat Intelligence Evidence** – number of reports, abuse confidence, and related reputation information.

These factors allow the system to evaluate an attacker using multiple pieces of evidence instead of depending on a single indicator.

7.2 Risk Score Calculation

Each observed factor contributes to the overall risk score. The individual values are combined to produce a final score representing the severity of the activity.

A simplified representation of the scoring process is:

$\text{Risk Score} = \text{Behaviour Score} + \text{IOC Reputation Score} + \text{Threat Intelligence Score} + \text{Activity Score}$

The final score is then mapped to a threat category such as:

Score Range	Threat Level
--------------------	---------------------

0–25	Low
------	-----

26–50	Medium
-------	--------

51–75	High
-------	------

76–100	Critical
--------	----------

The exact scoring weights can be adjusted according to the requirements of the deployment. This makes the engine flexible for different honeypot environments.

7.3 Explainability of the Score

An important feature of the engine is that the final score is not treated as a black-box result. The system can identify the major factors that contributed to the risk level.

For example, an IP may receive a high score because it:

- Generated a large number of failed login attempts.
- Repeatedly interacted with the honeypot.
- Executed suspicious commands.
- Had a poor external IP reputation.
- Was associated with multiple threat-intelligence reports.

This makes the result easier for an analyst to verify and understand.

7.4 Risk Classification and Prioritization

After calculating the score, the engine assigns a corresponding risk level. High-risk and critical events can be prioritized for further investigation, while low-risk events can be retained for monitoring.

The overall processing can be represented as:

HoneyPot Evidence → IOC Information → Threat Intelligence → Risk Factors → Risk Score → Threat Level → Explanation

This approach allows the platform to move from raw attacker activity to an actionable security assessment.

7.5 Role in the Overall Platform

The Explainable Risk Scoring Engine acts as the decision-making stage between threat-intelligence enrichment and security reporting. It combines locally observed attacker behaviour with external reputation information and produces a summarized risk assessment.

The engine therefore helps answer three important questions:

1. How risky is the observed activity?
2. Why was this risk level assigned?
3. Which evidence contributed to the score?

This improves the usefulness of the platform for security monitoring and investigation.

```
Select cowrie@ahira: ~/cowrie
(cowrie-env) cowrie@ahira:~/cowrie$ python automation/risk_engine.py

=====
EXPLAINABLE RISK ANALYSIS ENGINE
=====

Events loaded: 33
Source IPs found: 1

-----
RISK ASSESSMENT
-----
{
  "source_ip": "192.168.1.10",
  "risk_score": 44,
  "severity": "MEDIUM",
  "factors": {
    "successful_authentication": 20,
    "brute_force": 0,
    "reconnaissance": 9,
    "suspicious_command": 5,
    "payload_activity": 0,
    "persistence_attempt": 0,
    "repeated_activity": 10,
    "threat_intelligence": 0
  },
  "reasons": [
    "Successful authentication was observed",
    "Reconnaissance activity detected (9 commands)",
    "Suspicious command execution detected (1 commands)",
    "Repeated command activity detected (13 commands)",
    "Threat intelligence lookup skipped because source IP is private/non-public"
  ],
  "threat_intelligence": {
    "status": "SKIPPED",
    "message": "Private or non-public IP address"
  },
  "event_count": 33,
  "failed_login_attempts": 0,
  "command_count": 13
}

=====
RISK ANALYSIS COMPLETE
=====
(cowrie-env) cowrie@ahira:~/cowrie$
```

Figure 7: Explainable Risk Scoring Engine Output

8. INCIDENT CORRELATION & AUTOMATED REPORTING

The Incident Correlation & Automated Reporting module combines related security events and generates a summarized incident report. Instead of treating every honeypot event as an independent alert, the system correlates information such as source IP, timestamps, login attempts, commands, extracted IOCs, threat-intelligence results, and risk scores to identify related attacker activity.

8.1 Incident Correlation

The correlation process groups events that are associated with the same attacker or activity pattern. For example, multiple failed SSH logins followed by successful access and suspicious command execution can be treated as a single incident rather than several unrelated events.

The correlated information includes:

- Attacker/source IP
- Observed activity and timestamps
- Extracted IOCs
- Threat-intelligence reputation
- Calculated risk score and threat level
- Suspicious commands or behaviour

This provides a complete view of an attack and helps reduce unnecessary individual alerts.

8.2 Automated Reporting

After incidents are correlated and analyzed, the system generates a structured report containing the important details of the detected activity. The report can summarize the attacker information, observed behaviour, IOC details, threat-intelligence findings, risk level, and supporting evidence.

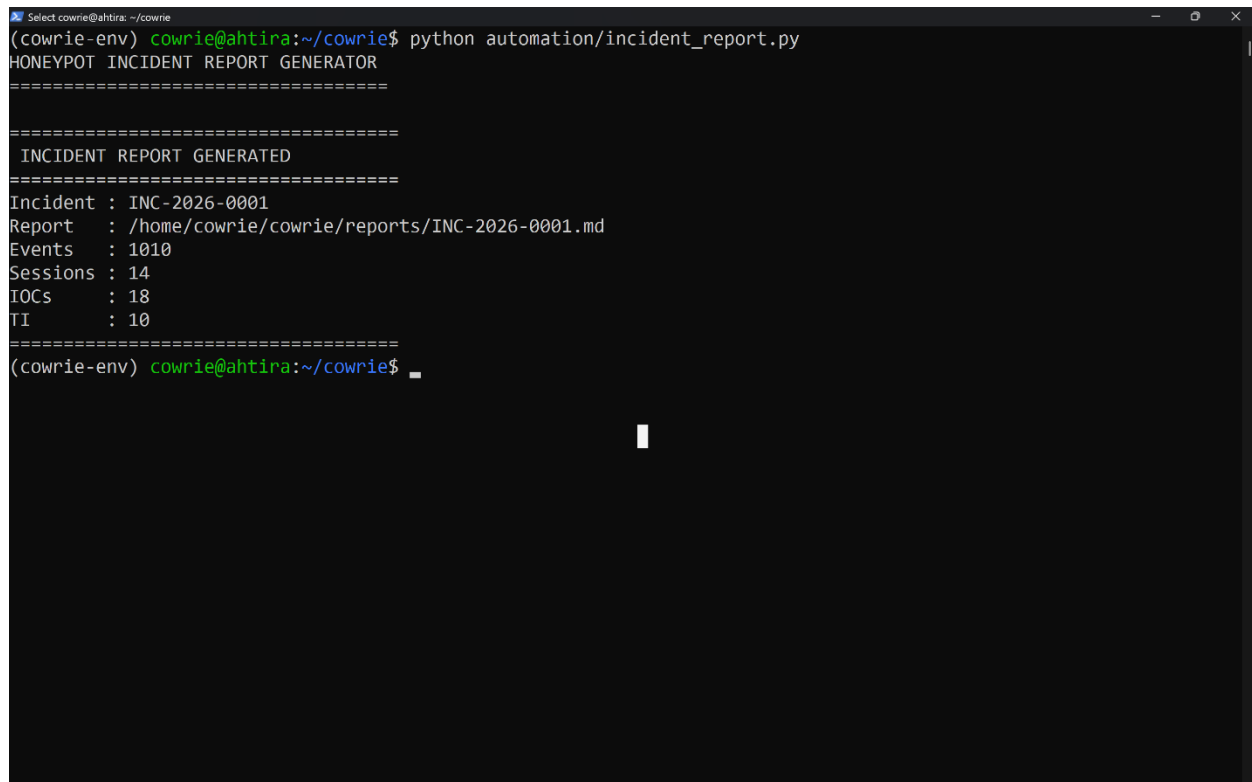
The overall workflow is:

Honeypot Events → Event Correlation → Incident Formation → Risk Information → Automated Report

Automating this process reduces manual analysis effort and provides a consistent format for documenting security incidents.

8.3 Implementation and Output

The incident correlation and reporting functionality uses the processed data generated by the earlier analysis stages. The resulting report provides security analysts with a concise summary of what happened, which source was involved, why it was considered suspicious, and how severe the incident is.

A terminal window titled 'Select cowrie@ahkira: ~/cowrie' shows the execution of a Python script. The prompt is '(cowrie-env) cowrie@ahkira:~/cowrie\$'. The command entered is 'python automation/incident_report.py'. The output is as follows:

```
(cowrie-env) cowrie@ahkira:~/cowrie$ python automation/incident_report.py
HONEYPOT INCIDENT REPORT GENERATOR
=====
INCIDENT REPORT GENERATED
=====
Incident : INC-2026-0001
Report   : /home/cowrie/cowrie/reports/INC-2026-0001.md
Events   : 1010
Sessions : 14
IOCs     : 18
TI       : 10
=====
(cowrie-env) cowrie@ahkira:~/cowrie$
```

Figure 8: Incident Correlation & Automated Reporting Output

9. SECURITY ANALYST DASHBOARD

The Security Analyst Dashboard provides a centralized interface for viewing and understanding the security information generated by the honeypot platform. Instead of checking raw logs and individual analysis outputs, the analyst can use the dashboard to quickly monitor attacker activity, detected IOCs, threat intelligence, risk levels, and correlated incidents.

9.1 Dashboard Information

The dashboard presents the most important security indicators in a summarized form, including:

- Number of detected attacks and events
- Top attacking IP addresses
- Extracted IOCs
- Threat-intelligence reputation
- Risk scores and threat levels
- High-risk or critical incidents
- Attacker activity and event trends

This allows the analyst to identify important events without manually examining every collected log.

9.2 Risk and Incident Visualization

Risk information can be displayed using charts, tables, counters, and status indicators. High-risk incidents can be highlighted so that they receive immediate attention. The dashboard can also provide information about frequently observed attackers and the types of activities recorded by the honeypot.

The dashboard acts as the final presentation layer of the platform, bringing together the results generated by the IOC extraction, threat-intelligence, risk-scoring, and incident-correlation modules.

9.3 Role in Security Monitoring

The dashboard improves the usability of the platform by converting complex analysis results into an easily understandable security overview. An analyst can use it to monitor the current threat situation, identify suspicious sources, investigate high-risk incidents, and review the evidence associated with detected activity.

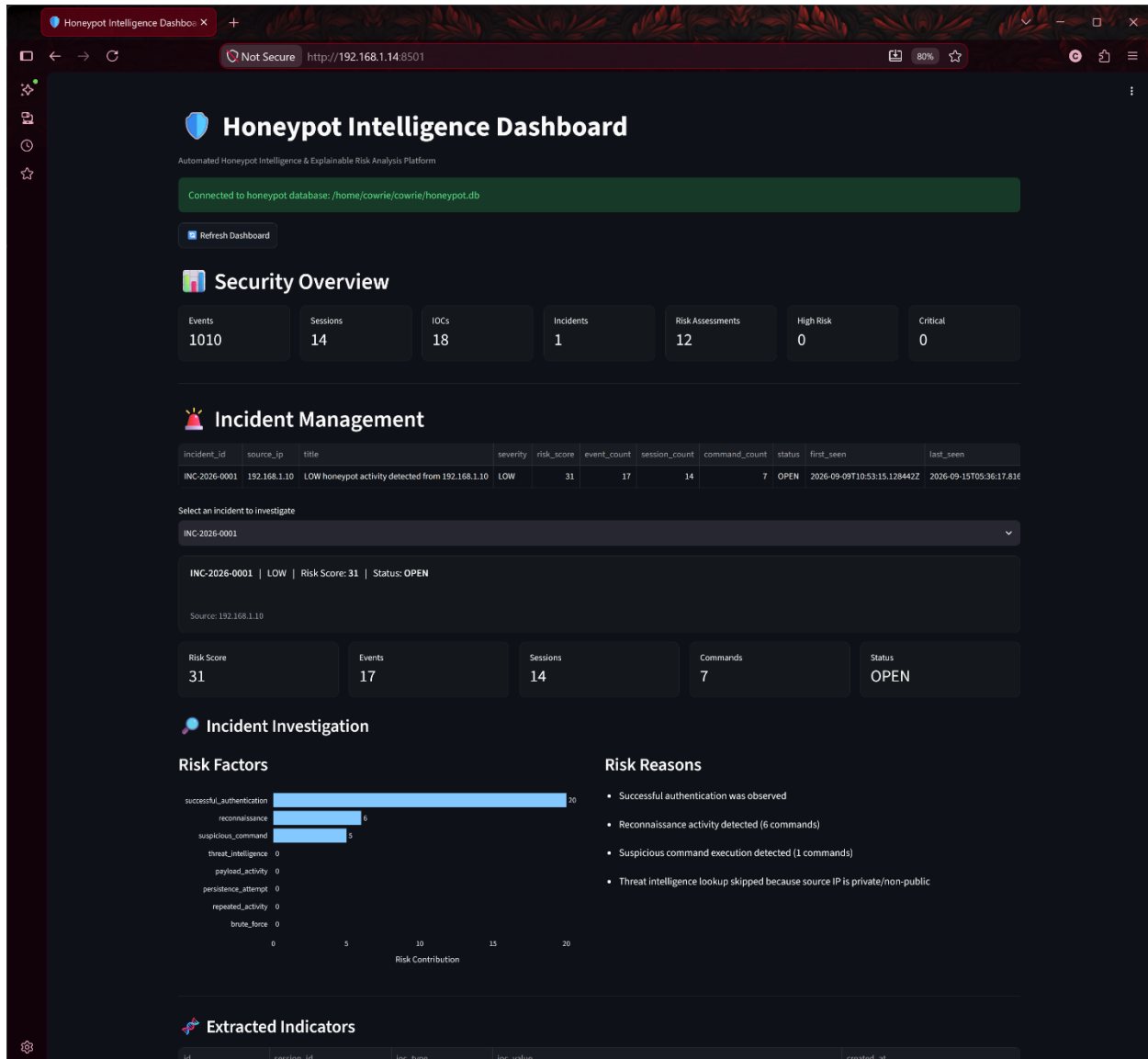


Figure 9.1: Security Analyst Dashboard – Security Overview and Incident Management

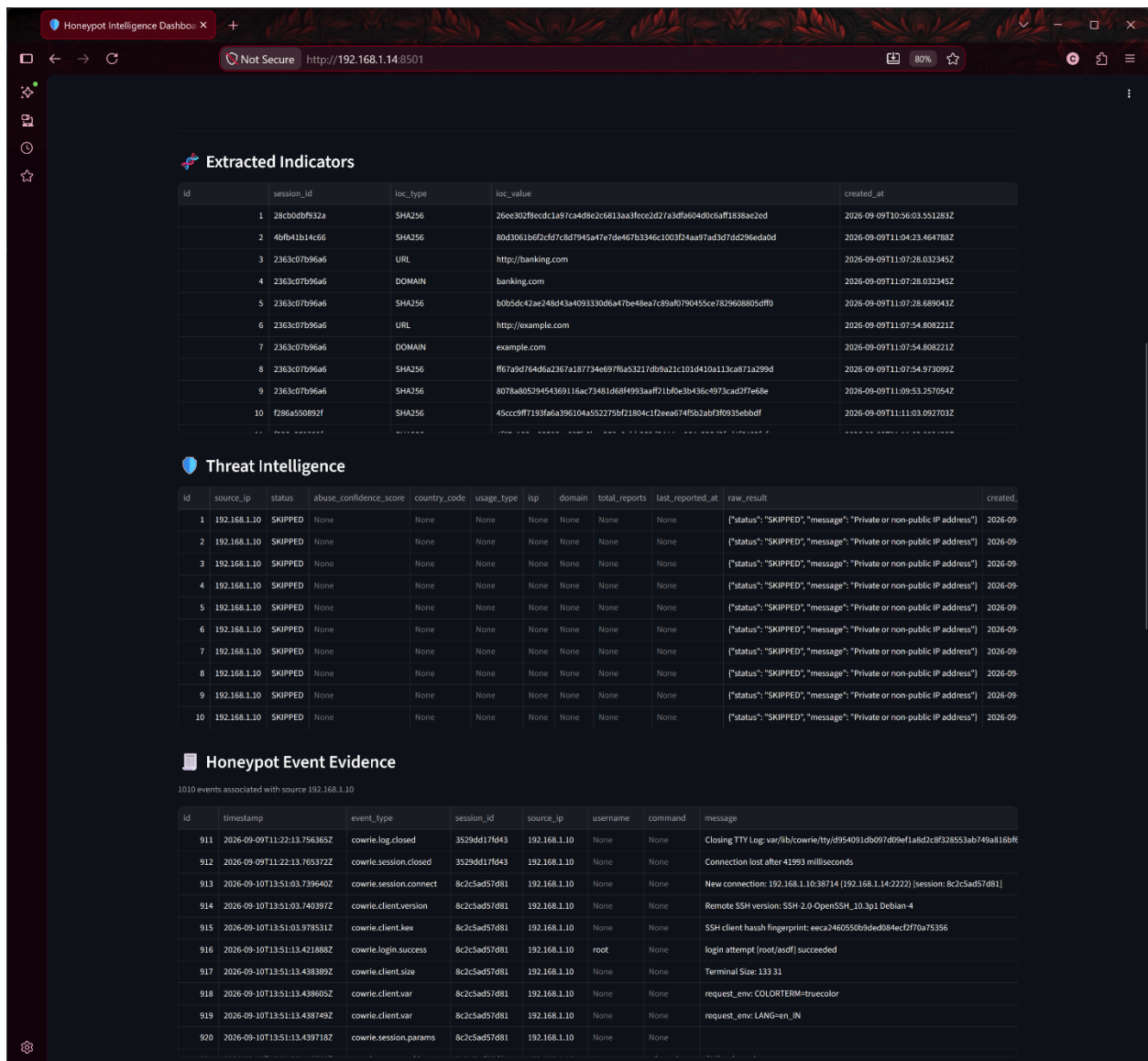


Figure 9.2: Security Analyst Dashboard – Incident Investigation and Risk Analysis

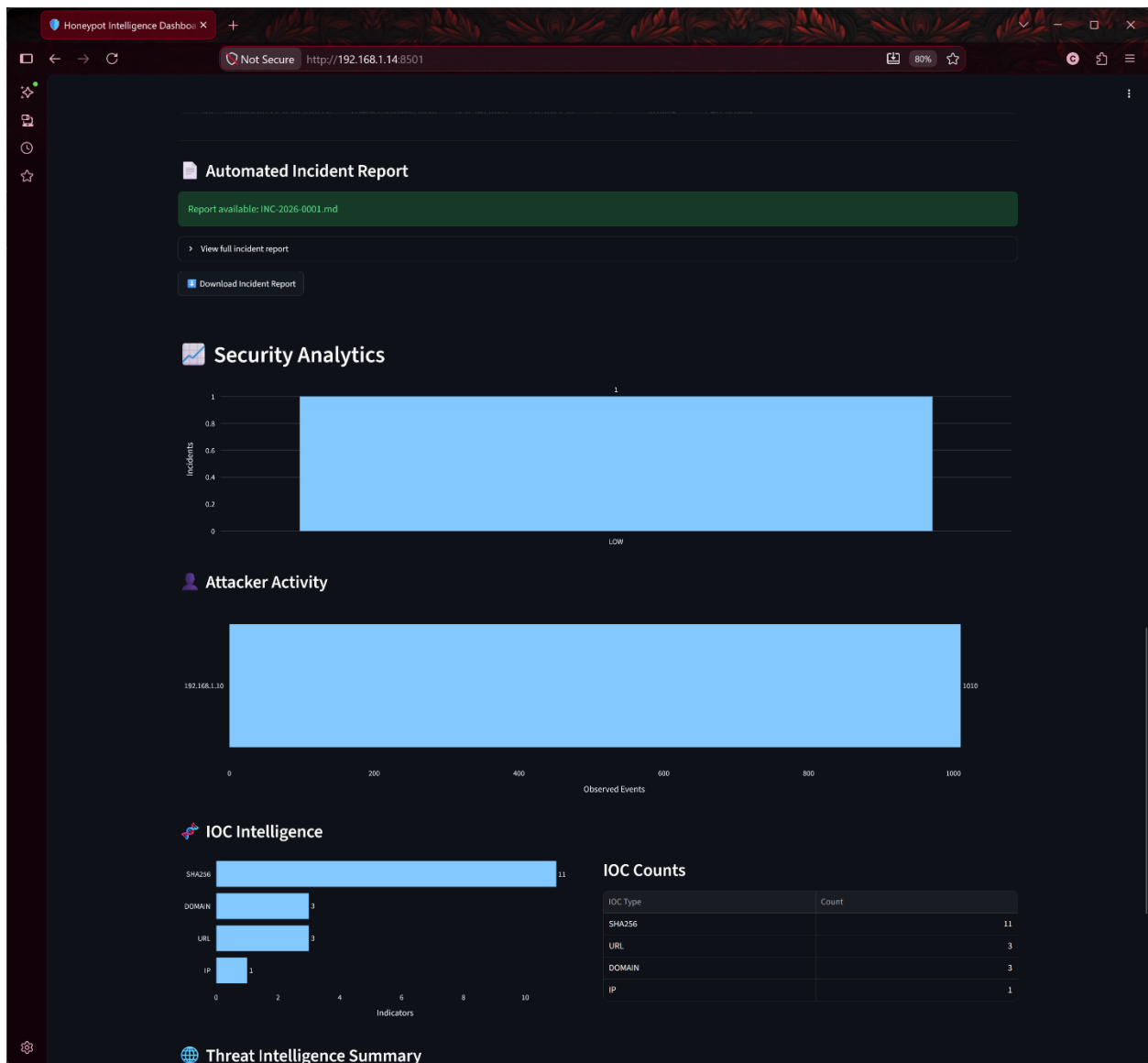


Figure 9.3: Security Analyst Dashboard – Automated Reporting and Security Analytics

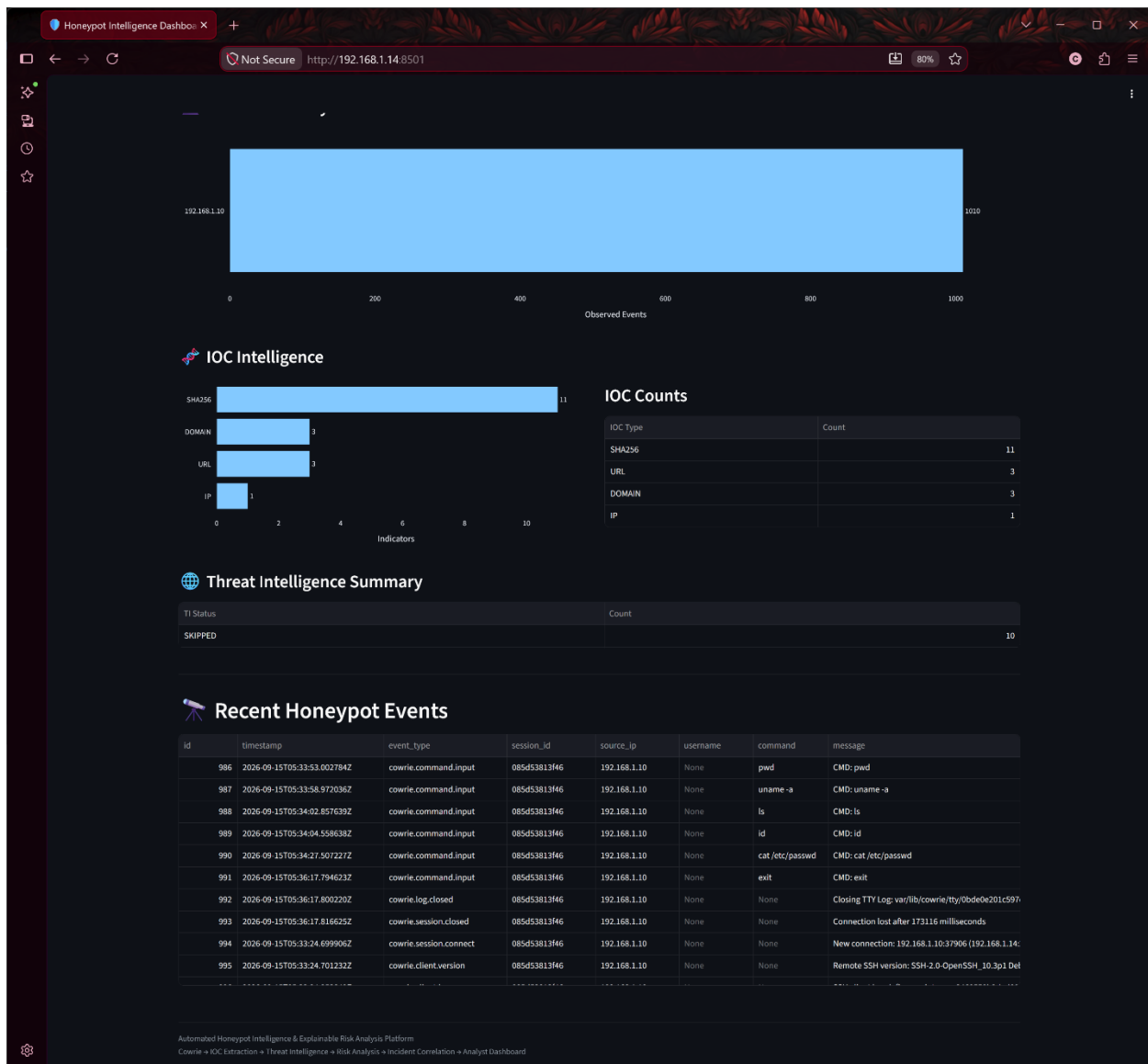


Figure 9.4: Security Analyst Dashboard – Threat Intelligence and Recent Honeypot Events

10. TESTING, VALIDATION & RESULTS

Testing was carried out to verify that the major components of the Automated Honeypot Threat Intelligence & Risk Analysis Platform work correctly together. The testing focused on the complete flow from honeypot data collection to IOC extraction, threat-intelligence enrichment, risk scoring, incident correlation, and dashboard presentation.

10.1 Functional Testing

Each major module was tested using relevant input data and its expected output was verified.

Module	Testing Performed	Expected Result
Honeypot Collection	Generate attacker activity	Events are recorded correctly
Log Processing	Process collected logs	Data is normalized successfully
IOC Extraction	Process suspicious events	IPs and other IOCs are extracted
Threat Intelligence	Query extracted IPs	Reputation information is retrieved
Risk Scoring	Process analyzed evidence	Risk score and threat level are generated
Incident Correlation	Combine related events	Related activities are grouped
Dashboard	Display processed results	Security information is shown clearly

10.2 Validation

The outputs from each stage were compared with the expected behaviour of the system. Extracted IOCs were checked against the original honeypot logs, while threat-intelligence results were verified against the returned API information. Risk scores and incident classifications were also checked to ensure that the generated results were consistent with the available evidence.

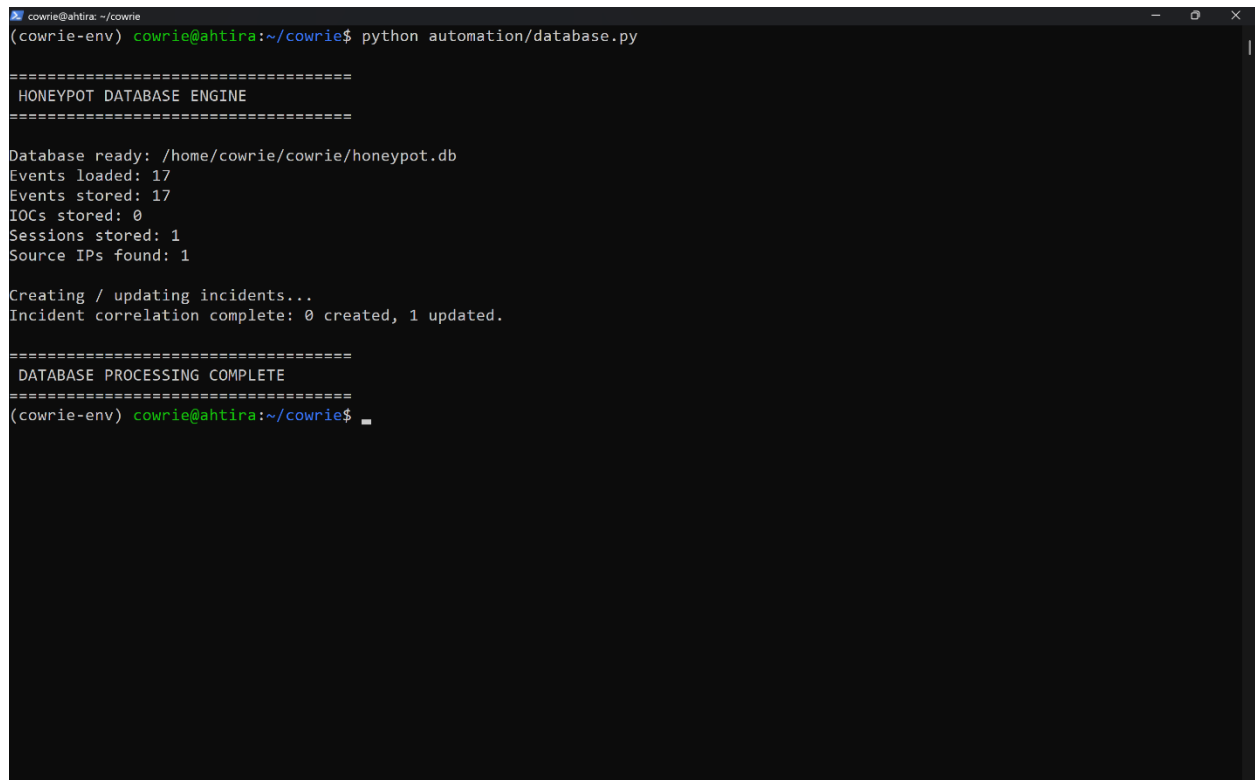
The complete pipeline was validated using the following flow:

Honeypot Logs → Processing → IOC Extraction → Threat Intelligence → Risk Scoring → Incident Correlation → Dashboard

Successful execution of this pipeline confirms that the individual modules can operate together as a unified platform.

10.3 Results

Testing demonstrated that the platform can successfully collect attacker activity, extract useful security indicators, enrich them with external intelligence, calculate an explainable risk level, correlate related events, and present the results through the analyst dashboard.



```
cowrie@ahtra: ~/cowrie
(cowrie-env) cowrie@ahtra:~/cowrie$ python automation/database.py

=====
HONEYPOT DATABASE ENGINE
=====

Database ready: /home/cowrie/cowrie/honeypot.db
Events loaded: 17
Events stored: 17
IOCs stored: 0
Sessions stored: 1
Source IPs found: 1

Creating / updating incidents...
Incident correlation complete: 0 created, 1 updated.

=====
DATABASE PROCESSING COMPLETE
=====
(cowrie-env) cowrie@ahtra:~/cowrie$
```

Figure 10: Database Processing and Incident Correlation Validation Output

11. CONCLUSION

The Automated Honeypot Threat Intelligence & Risk Analysis Platform successfully provides an automated approach for collecting, processing, and analysing suspicious activities captured through a honeypot environment. The system converts raw telemetry and log data into structured security information through log processing, IOC extraction, behavioural analysis, and threat-intelligence enrichment.

The integration of external threat intelligence improves the analysis by providing additional reputation information for observed indicators. The Explainable Risk Scoring Engine further combines attacker behaviour and intelligence data to produce a meaningful risk level while providing the reasons behind the classification. This makes the generated results easier for a security analyst to understand and investigate.

The Incident Correlation and Automated Reporting components help organize related events into meaningful security incidents, while the Security Analyst Dashboard provides a centralized view of attacks, IOCs, risk levels, and other important findings. Testing and validation confirmed that the major modules can work together as a complete analysis pipeline.

Overall, the project demonstrates how honeypot telemetry, threat intelligence, automated risk assessment, incident correlation, and visualization can be combined into a practical security monitoring platform. The system can be further extended with additional threat-intelligence sources, more advanced behavioural analysis, automated notifications, and machine-learning-based detection to improve its capabilities.

12. REFERENCES

1. **Cowrie Honeypot Documentation** — Installation, configuration, and honeypot deployment reference.
[Cowrie Documentation](#)
2. **AbuseIPDB API Documentation** — API-based IP reputation and abuse information.
[AbuseIPDB API Documentation](#)
3. **MITRE ATT&CK** — Knowledge base of adversary tactics and techniques.
[MITRE ATT&CK](#)
4. **NIST Cybersecurity Framework 2.0** — Cybersecurity risk management and security outcomes.
[NIST Cybersecurity Framework 2.0](#)
5. **NIST SP 800-61 Rev. 3** — Incident response recommendations and considerations.
[NIST Incident Response Guidance](#)
6. **OWASP Logging Cheat Sheet** — Security logging, event collection, and log management guidance.
[OWASP Logging Cheat Sheet](#)
7. **CISA Cybersecurity Resources** — Cyber threat information, advisories, and defensive guidance.
[CISA Cybersecurity Resources](#)
8. **MISP – Malware Information Sharing Platform** — Threat intelligence sharing and IOC management platform.
[MISP Project](#)
9. **OASIS STIX** — Structured Threat Information Expression standard for representing cyber threat intelligence.
[OASIS STIX Standard](#)
10. **MITRE D3FEND** — Knowledge base for cybersecurity defensive techniques.
[MITRE D3FEND](#)

Appendix A – Contact Information

Project Team

Rahul Reghu Rajan

Cybersecurity | CEH Certified | SOC & VAPT | Threat Detection | Penetration Testing |
Information Security

Email: rahulreghurajan@gmail.com

Portfolio: rahulhub.vercel.app

LinkedIn: linkedin/rahul-reghu-rajan

GitHub: [github/RAHUL-REGHU-RAJAN](https://github.com/RAHUL-REGHU-RAJAN)

Binny Hinn

AI/ML Enthusiast | Python Developer | LLMs & Generative AI | AI Automation | Cloud
& Backend Development

Email: binnyhinnr@gmail.com

LinkedIn: linkedin/binnyhinnr

GitHub: [github/binnyhinn](https://github.com/binnyhinn)